Sampling and Per-Precision Histograms with nixnan

An Experiment on `rd_nixnan.cu`

ganesh@cs.utah.edu

September 23, 2026

The Example, at a Glance

`rd_nixnan.cu`: a 1D reaction-diffusion PDE $\partial u/\partial t = D\,\partial^2 u/\partial x^2 + \lambda u$, explicit FTCS, 101-point grid, 1600 steps — the *same* kernel implemented once each in FP16, BF16, FP32, FP64.

How precision-sensitive is this example?

- The reaction term grows every value by $\sim$1.05$\times$ per step, *regardless of format* — so the only thing that differs between formats is how much exponent range each one has before it overflows.
- FP16's 5-bit exponent runs out at step 300; BF16/FP32/FP64's wider exponents are all still finite at step 1600 — same algorithm, same input, one format fails and three don't.

Concept 1: Sampling — The Mechanism

Every run here uses `nixnan.so` directly ("straight nixnan"), no wrapper:

```
LD_PRELOAD=<path>/nixnan.so [ENV_VARS...] ./rd_nixnan
```

`SAMPLING=N` instruments only every *N*th launch of a repeated kernel — a knob trading overhead for observation completeness.

Verified before trusting it: an older comment in this repo claims *"SAMPLING is a no-op."* We re-tested directly (`SAMPLING=1` vs. `SAMPLING=100000`): exception counts dropped to exactly zero at the high value. **Confirmed fixed** in the current build.

Concept 1: Every Call vs. Once in 8

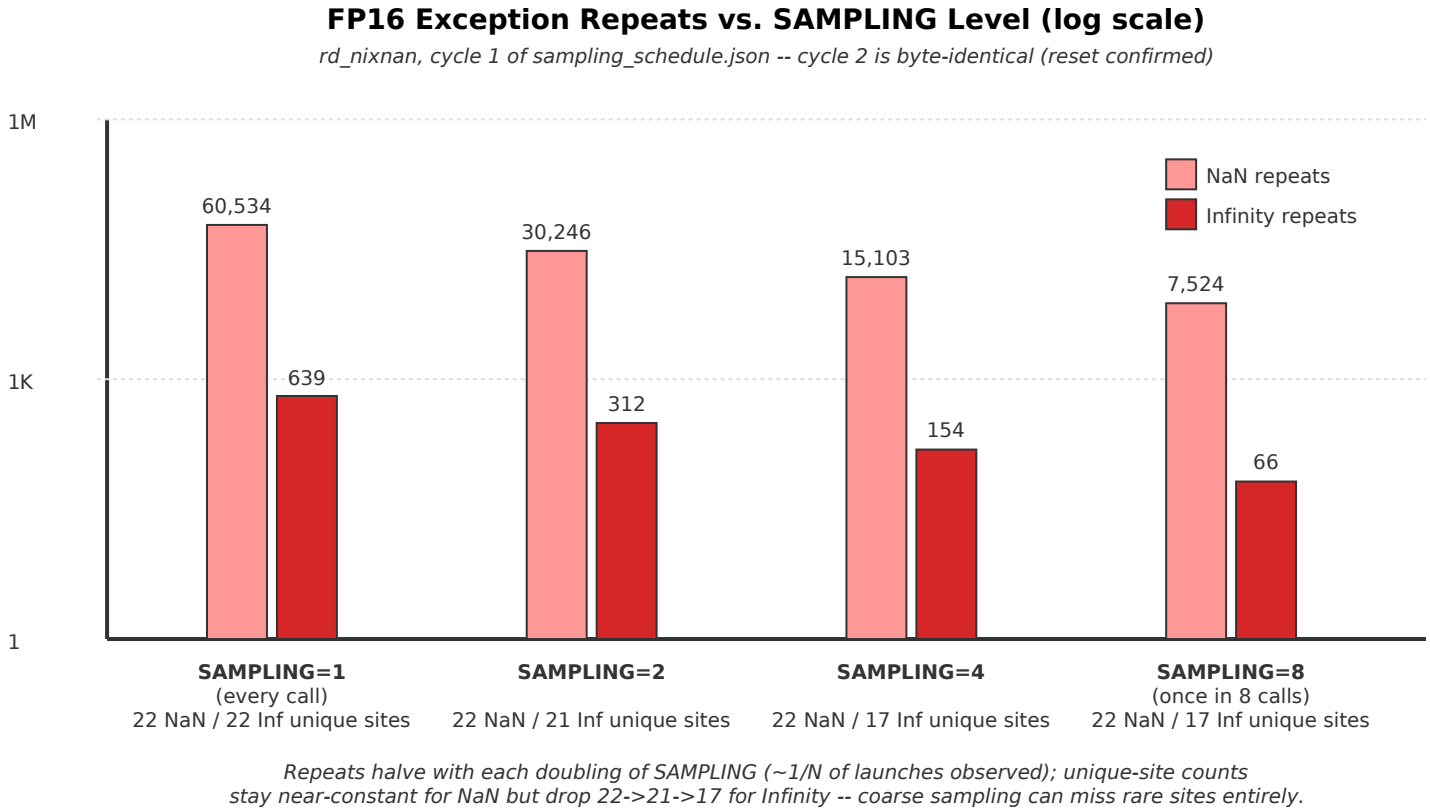
FP16 exception counts (the only format that overflows here):

	NaN (unique / repeats)	Infinity (unique / repe
`SAMPLING=1` (every call)	22 / 60,534	22 /
`SAMPLING=8`	22 / 7,524	17 /

Repeats drop $\sim 8\times$, as expected. But **5 distinct Infinity sites vanish entirely** ($22 \rightarrow 17$) — coarse sampling doesn't just under-count, it can miss whole bugs.

Concept 1: An Exponentially Decaying Schedule

`SAMPLING` is fixed for a whole process run, so a decaying rate is built as a sequence of short runs (`sampling_schedule.json`): step 4 times at spacing 1 $\rightarrow$ double to 2 $\rightarrow$ 4 $\rightarrow$ 8 $\rightarrow$ **reset**.



Every snap at a level is byte-identical; **cycle 2 exactly reproduces cycle 1** — the reset is provably clean.

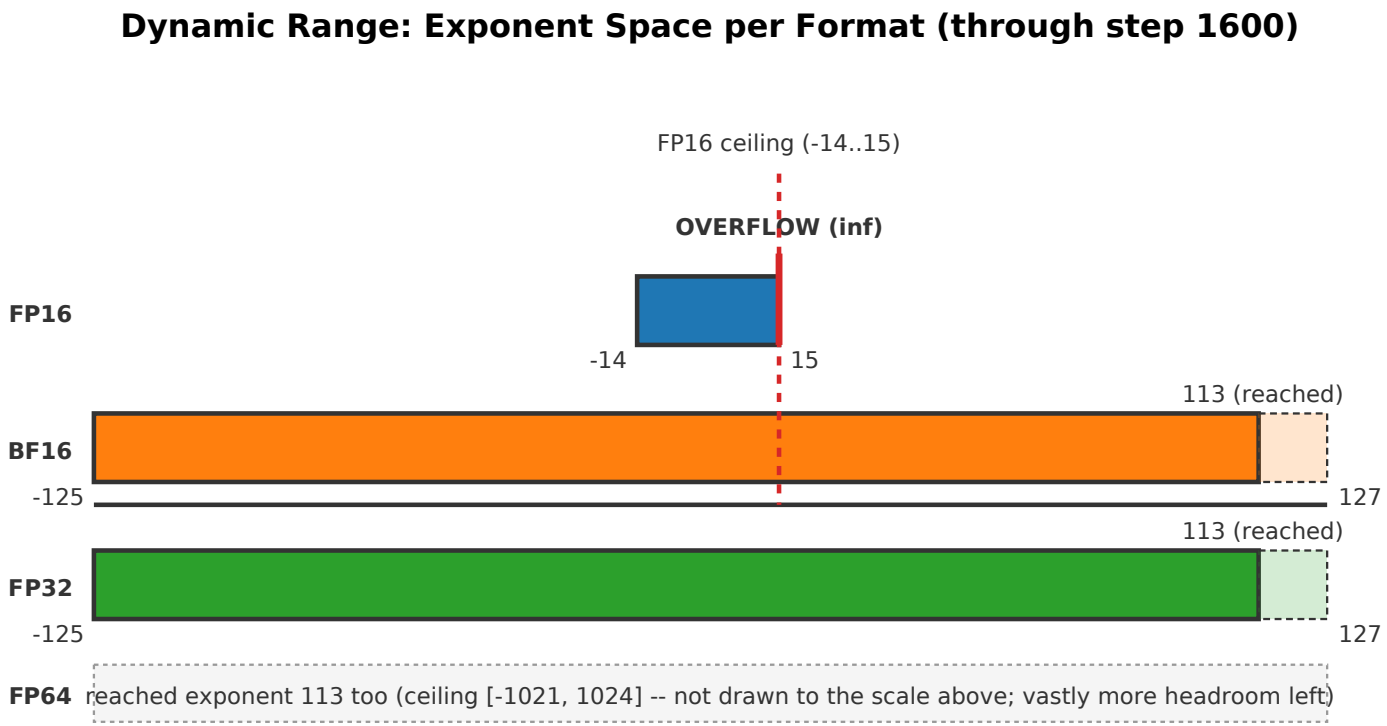
Concept 2: Histograms per Precision — The Mechanism

`HISTOGRAM=1` tracks the exponent of every instrumented FP value, independent of exception checking:

```
HISTOGRAM=1 LD_PRELOAD=nixnan.so ./rd_nixnan
```

At exit, nixnan reports the [min, max] exponent *actually observed* for each format — one number per precision bucket, no per-value log needed.

Concept 2: Results — Which Format Actually Failed



`f16` alone reports `inf`. `bf16`/`f32`/`f64` all reach the *identical* exponent, 113 — same trajectory, different ceilings, only one ceiling reached.

What nixnan Made Visible Here

1. **Sampling trades overhead for completeness**, not just verbosity — coarse sampling can drop entire exception sites, not only reduce their reported repeat count.
2. **A schedule can be built around a static env var** via orchestration, and — because the program is deterministic — that schedule's reset is independently verifiable.
3. **Per-format exponent histograms turn "FP16 probably fails" into a specific, quantified claim**: FP16 alone overflows; the other three formats are proven to share one common growth trajectory up to a shared checkpoint exponent.

Bonus: re-verifying SAMPLING here also caught that a previously documented nixnan regression has since been fixed.